Università degli Studi di Trento Corso di Programmazione I – E3: 145935 3° Appello – 14/06/2021 – a.a. 2020/2021	**PC N°**
COGNOME ..... **NOME** ..... **MATRICOLA**	
RITIRATO ☐ **FIRMA PER RITIRO**.....	

Esame online programmazione 1

PARTE TEORICA: 16 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Teo01 – punti 5]
[TEO] Scrivere una funzione ***filtraDati*** che preso in input in modo opportuno un array di valori reali determini la **media** e restituisca un array contenente i soli valori presenti nell'array passato che sono maggiori della media calcolata. Per il dato ritornato si consiglia di creare un opportuno tipo di dato ad hoc.
Definire i parametri passati alla funzione e mostrare l'invocazione della funzione dal main passando i parametri corretti ed inizializzati e stampare sempre nel main, in modo opportuno, i valori ritornati contenuti nell'array.
Esempio: se l'array passato come parametro contenesse i seguenti valori:
`{ 8.8, 0.3, -3.7, 9.1, -4.1 }`
la funzione ritornerebbe un array contenente `{ 8.8, 9.1 }` essendo la media pari a `2.08`

[Teo02 – punti 2]
La condizione ((A || B) && C):
☐ Vero ☐ Falso è sicuramente vera se C è vera
☐ Vero ☐ Falso è sicuramente falsa se B è falsa
☐ Vero ☐ Falso può essere vera anche se C è falsa
☐ Vero ☐ Falso è sicuramente falsa se sia A sia B sono false

Un parametro passato come input ad una funzione:
☐ Vero ☐ Falso non può mai essere modificato dalla funzione
☐ Vero ☐ Falso può essere modificato, se passato per indirizzo
☐ Vero ☐ Falso può essere modificato, purché passato per valore
☐ Vero ☐ Falso può essere modificato se non è passato in modo const

La differenza tra un ciclo while e un ciclo do-while è che:
☐ Vero ☐ Falso il ciclo do-while termina sempre
☐ Vero ☐ Falso nel ciclo while le istruzioni sono eseguite almeno una volta
☐ Vero ☐ Falso non c'è nessuna differenza
☐ Vero ☐ Falso nessuna delle precedenti

Un algoritmo:
☐ Vero ☐ Falso può solo essere scritto in un linguaggio di programmazione
☐ Vero ☐ Falso ha sempre un numero finito di istruzioni
☐ Vero ☐ Falso deve essere specificato in un linguaggio naturale
☐ Vero ☐ Falso deve essere comprensibile a un calcolatore

[Teo03 – punti 3]
[TEO] Definire una funzione che permette al chiamante di decrementare di uno il valore di una variabile intera passata come paramentro.

Definirne due versioni, "**dec_byvalue**", che adotta il passaggio dei parametri per valore, e "**dec_byref**", che adotta il passaggio per riferimento.
Definire inoltre la funzione **main** in cui vengono richiamate le due funzioni e stampato il risultato finale.

[Teo04 – punti 2]
[TEO] Spiegare in dettaglio che differenze ci sono tra le seguenti modalità di apertura di un canale con un file: "**w**" e "**wb**" "**a**" e "**ab**".

[Teo05 – punti 4]
[TEO] [a] Dato un albero binario BST definire cosa si intende con percorso.
[b] Scrivere inoltre una funzione ricorsiva **`TreeMinDepth`** per calcolare il percorso radice-foglia più breve in un BST.
Si suppone che la struttura sia definita come segue:

```
typedef struct Tnode {
  float data;
  Tnode* left;
  Tnode* right;
} Tnode;
```

```
int TreeMinDepth(Node* p){
  return (p==NULL)?0:min(TreeMinDepth(p->left), TreeMinDepth(p->right))+1;
}
```

PARTE PRATICA: 16 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Cod-punti 16] (IDEWEB1: www.programiz.com/cpp-programming/online-compiler/ IDEWEB2: http://cpp.sh/) Materiale: http://www.cplusplus.com/reference/)
[COD] Date le seguenti strutture

```
typedef enum TipoIntervento{ ACQUA, INCENDIO, INCIDENTE } TipoIntervento;
typedef struct {
  char localita[20];
  unsigned int level;
  TipoIntervento ti;
  Tintervento ()  { /* Tintervento da implementare */ }
  void stampa() { /* da implementare → formato: vedi punto D */ }
} Tintervento;
typedef struct TInterventiVF{ //elemento base di lista LIFO
  Tintervento intervento;
  TInterventiVF *next;
  TInterventiVF () { /* da implementare */ }
  TInterventiVF (Tintervento v, TInterventiVF *n) { /*da implementare*/}
  void stampa() { /* da implementare */ }
} TInterventiVF;
```

[ParteA – punti 4] Completare nelle definizioni delle strutture i metodi, costruttori indicati come "da implementare" ed aggiungere eventuali altri metodi/costruttori ritenuti necessari e/o utili. Modificare e completare inoltre il **main** inserendo il seguente codice:

```
TInterventiVF* interventiVF[3]; //array di liste LIFO
/* inizializzare array interventiVF in modo opportuno */
Tintervento interv;
for(int i=0; i<5; i++) {
  nuovoIntervento(&interv);
  /* in funzione del valore del campo "ti" della variabile interv inserire
  in posizione corretta dell'array interventiVF il valore inizializzato
  ACQUA->0 INCENDIO->1, INCIDENTE->2, per esempio se ti=INCENDIO usare:
```

```
    interventiVF[1] = insIntervento(interventiVF[1], interv);
    */
  }
  stampaInterventi(interventiVF);
  salvaInterventi(interventiVF);
```

[ParteB – punti 3] Implementare una funzione **nuovoIntervento** che riceve in input per riferimento una variabile di tipo **Tintervento interv** e la inizializza con:

- campo **ti** valore casuale tra ACQUA, INCENDIO, INCIDENTE;
- campo **level** valore casuale tra 1 e 9 (inclusi).
- campo **localita** letto da tastiera.

[ParteC – punti 3] Implementare una funzione **insIntervento** che riceve come input una variabile di tipo **TInterventiVF* intvf** (lista) e variabile di tipo **Tintervento interv** ed inserisce l'intervento **interv** nella lista **intvf** (*push*). La funzione restituisce l'indirizzo della lista aggiornata.

[ParteD – punti 3] Implementare una funzione **stampaInterventi** che riceve in input un array di tipo **TInterventiVF* allintvf** (array di liste LIFO) e stampa il contenuto delle liste utilizzando opportunamente il metodo **stampa** di **Tintervento**. Nella forma (per l'enum stampare etichetta in modo opportuno):

localita **TIPO=**ti **LEVEL=**level

Esempio: Milano **TIPO=**INCENDIO **LEVEL=**7

[ParteE – punti 3] Implementare una funzione **salvaInterventi** che riceve in input un array di tipo **TInterventiVF* allintvf** (array di liste LIFO) e scorre tutti gli elementi salvandoli su un file denominato "**interventi.txt**", nella forma:

localita **TIPO=**ti **LEVEL=**level